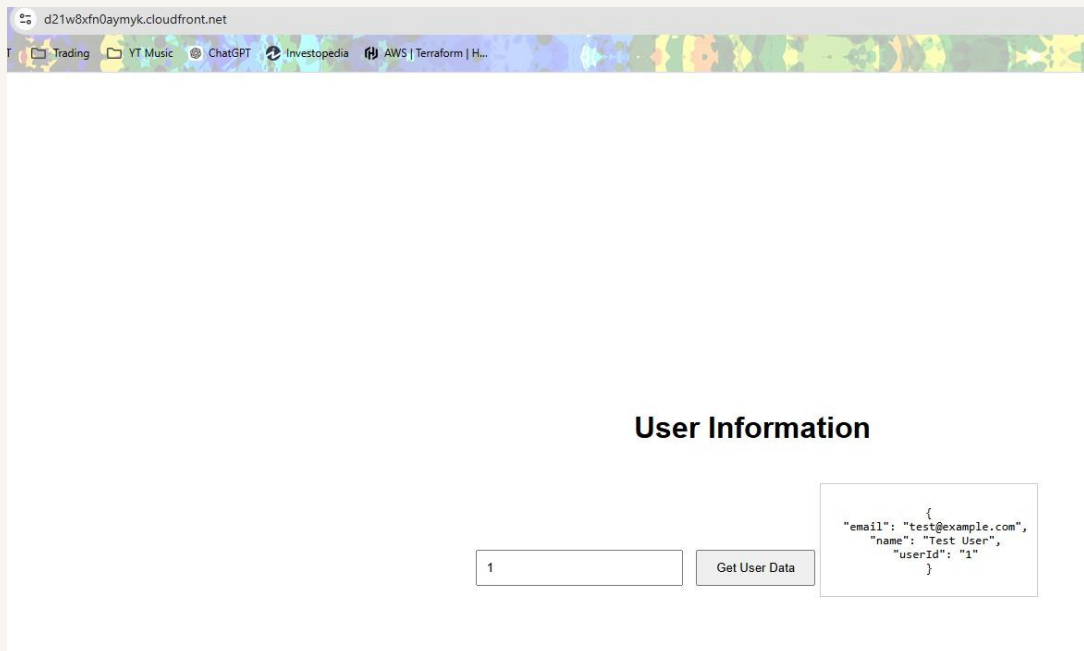


Build a Three-Tier Web App

Melvin green



Introducing Today's Project!

In this project, I will demonstrate my ability to build my own three-tier app. I'm doing this project to learn how to create all three tiers: the presentation tier, the logic tier, and the data tier.

Tools and concepts

Services I used were CloudFront, S3, API Gateway, Lambda, and DynamoDB. Key concepts I learned include Lambda functions, JSON, CORS, S3 bucket policies, IAM policies, REST APIs, Origin Access Control (OAC), CloudFront distributions and caching (including cache invalidation), the Default Root Object setting, Lambda proxy integration, using browser DevTools to debug errors, and how the three-tier architecture (presentation, logic, and data tiers) connects together.

Project reflection

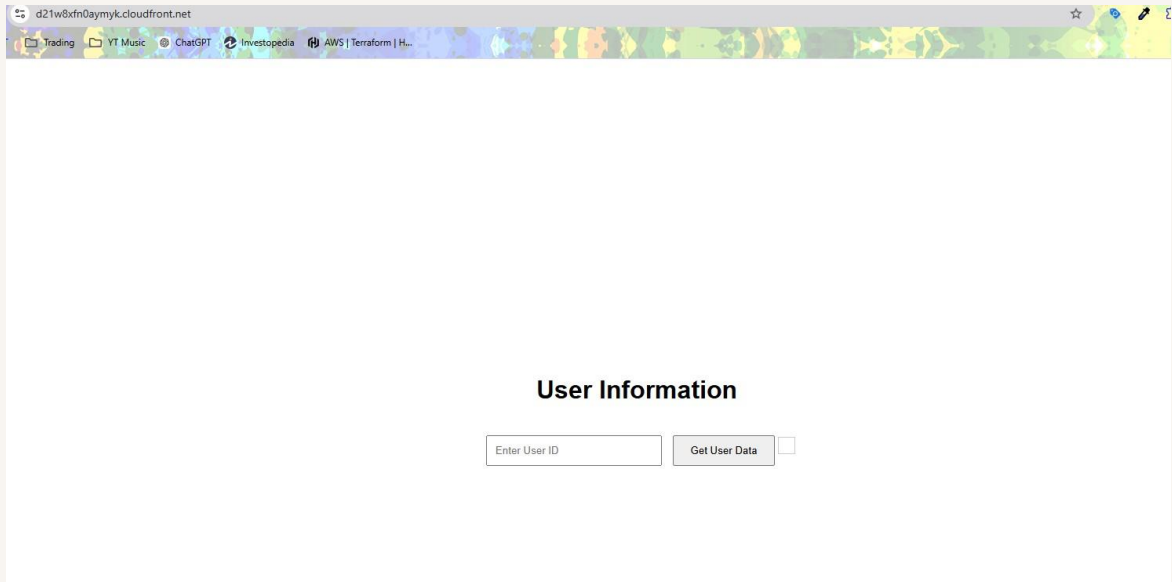
This project took me approximately two and a half hours. The most challenging part was connecting my S3 bucket to my CloudFront distribution. It was most rewarding to connect API Gateway with the Lambda function in order to make the website work properly.

I chose to do this project today because I wanted to put all three tiers together and see how they work properly.

Presentation tier

For the presentation tier, I will set up an S3 bucket and upload an index.html file. I will then set up CloudFront, since the S3 bucket will store my website files and CloudFront will deliver my website content globally.

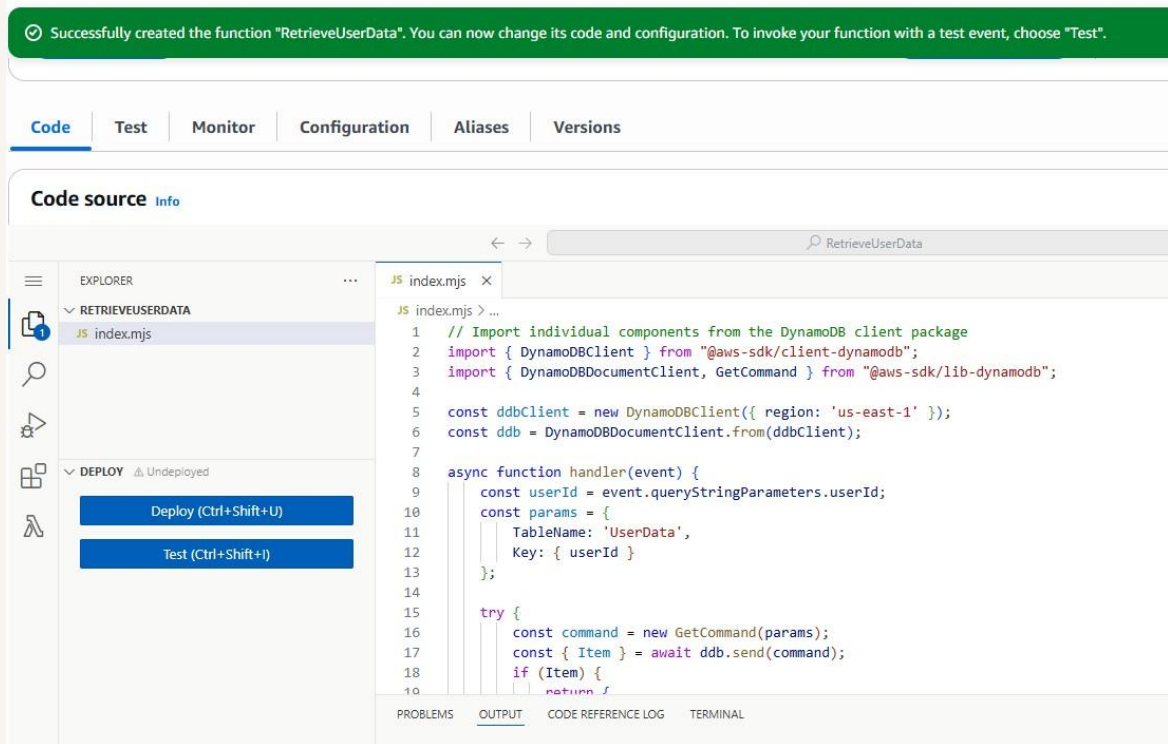
I accessed my deployed website by setting up a policy in the S3 bucket policy to grant access to CloudFront. Once that was created, I validated that the content was being delivered by copying the Distribution Domain Name URL and checking it via the browser.



Logic tier

For the logic tier, I will create a Lambda function to fetch data from the DynamoDB table. Following that, I will write the code for my Lambda function. Next, I will create an API Gateway REST API, create a resource and method to handle the GET request, and deploy the API to make it accessible.

The Lambda function retrieves data using triggered events, such as an HTTP request or a change in the S3 bucket. When the event is triggered, the Lambda function executes the code in the runtime environment I picked.



Data tier

For the data tier, I will set up a DynamoDB table and add user data to my table. The data tier is where I will store all of the data that my application uses.

The partition key for my DynamoDB table is how the data is organized in DynamoDB. I think of it as a label that can be used to group similar items. The partition key is how DynamoDB spreads out data across different servers for quick access and efficient querying.

Create item

You can add, remove, or edit the attributes of an item. You can nest attributes i

Attributes

☐ View DynamoDB JSON

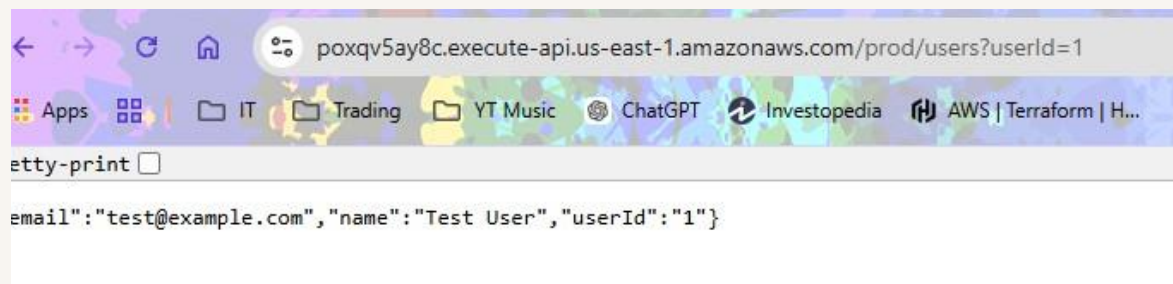
```
1 {  
2   "userId": "1",  
3   "name": "Test User",  
4   "email": "test@example.com"  
5 }
```

6

Logic and Data tier

Once all three layers of my three-tier architecture are set up, the next step is to update my `index.html` file to make a request to the API Gateway endpoint and display the returned data. I will update the `script.js` file with JavaScript code to make an API request and verify that the data is displayed on my website.

To test my API, I copied the Invoke URL and added `/users?userId=1` to the end of the URL and ran it in the browser. The results displayed the table's data being returned by the API.



Console Errors

The error in my distributed site was because there was an error in my script.js. I was able to find this by using the F12 function in my browser after performing the action. The console showed me where the exact problem was in my script.

To resolve the error, I updated script.js by replacing the placeholder in the script with the Invoke URL. I then re-uploaded it to S3 because I needed to add the updated file to the S3 bucket.

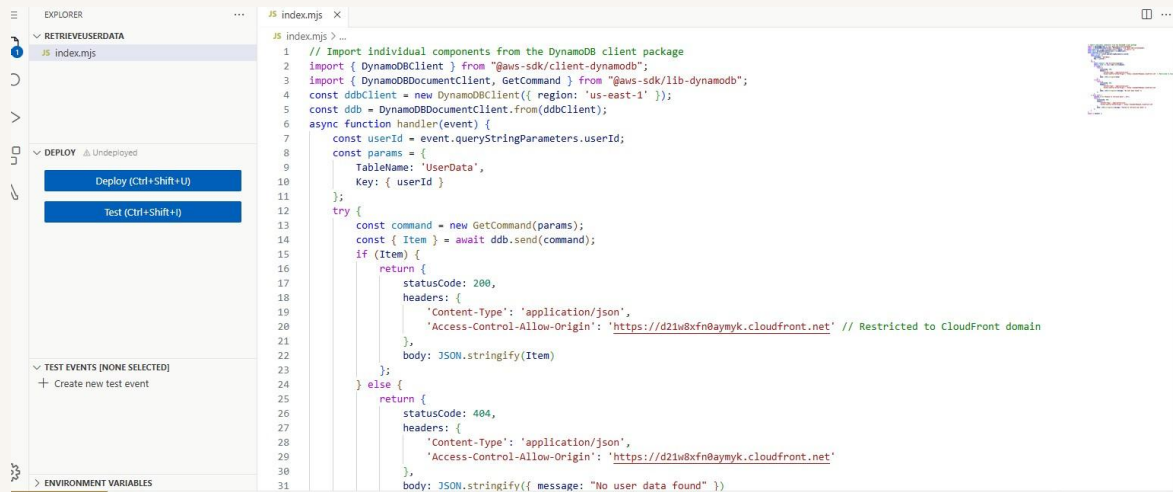
I ran into a second error after updating script.js. This was an error with CORS (Cross-Origin Resource Sharing). This error is happening because my API Gateway is not configured to allow requests from my CloudFront URL.



Resolving CORS Errors

To resolve the CORS error, I first went to my API resource tab. Then I clicked on the /users resource and enabled CORS. Once I did that, I added GET and OPTIONS under Access-Control-Allow-Methods. After that, I added my CloudFront Distribution Domain Name as the Access-Control-Allow-Origin. This should allow requests from the CloudFront domain to my API.

I also updated my Lambda function because I'm using Lambda proxy integration, which means API Gateway passes my Lambda's response straight through without modifying it. This means the CORS headers must be added in my function's response rather than through API Gateway's CORS settings. It is best practice to replace * with the CloudFront domain rather than allowing all origins.



```
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
4 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
5 const ddb = DynamoDBDocumentClient.from(ddbClient);
6 async function handler(event) {
7   const userId = event.queryStringParameters.userId;
8   const params = {
9     TableName: 'UserData',
10    Key: { userId }
11  };
12  try {
13    const command = new GetCommand(params);
14    const { Item } = await ddb.send(command);
15    if (Item) {
16      return {
17        statusCode: 200,
18        headers: {
19          'Content-Type': 'application/json',
20          'Access-Control-Allow-Origin': 'https://d21w8xfn8aymyk.cloudfront.net' // Restricted to CloudFront domain
21        },
22        body: JSON.stringify(Item)
23      };
24    } else {
25      return {
26        statusCode: 404,
27        headers: {
28          'Content-Type': 'application/json',
29          'Access-Control-Allow-Origin': 'https://d21w8xfn8aymyk.cloudfront.net'
30        },
31        body: JSON.stringify({ message: "No user data found" })
32      };
33    }
34  }
35}
```

Fixed Solution

I verified the fixed connection between API Gateway and CloudFront by adding my CloudFront domain to the Access-Control-Allow-Origin header in my Lambda function, then reloading my website and checking the browser console (F12) to confirm the CORS error was gone and the user data was displayed correctly.

